Distributed Tic-Tac-Toe: Technical Report

Alexandru Eduard Toni

October 24, 2026

1 Analysis & Requirements

The goal of this project is to implement a distributed, concurrent multiplayer Tic-Tac-Toe game using **Java RMI (Remote Method Invocation)** for network communication and **JavaFX** for the graphical user interface.

- **Match Creation:** A player can create a new game with a name of their choice and wait for an opponent. Creating a game with a name that is already in use must fail.
- **Match Joining:** A player can join an existing game, given its name. Joining must fail if no game with that name exists or if the game already has two players.
- **Turn-Based Concurrency:** The server must handle multiple concurrent games safely, ensuring players take alternating turns. Requests of different clients are executed by different RMI threads, so every operation on a match must be free of race conditions.
- **Real-time Notifications:** The server must broadcast game events (start, moves, game over) asynchronously to all participants in a match using RMI callbacks. A client that becomes unreachable must not block or corrupt the match.

2 System Architecture & Design

The system follows a distributed client-server architecture based on the **distributed object** model of RMI: the server exports a remote object that clients invoke through a stub (**Remote Procedure Call**), and each client exports a callback object that the server invokes to push events (**Observer Pattern**).

2.1 Class Structure (UML Overview)

- **GameEngine (Interface):** Remote interface exposed by the server, with three operations: `createGame()`, `joinGame()` and `makeMove()`.
- **GameEngineImpl:** Server-side remote object registered in the RMI registry. It only keeps the directory of the active matches (a map from match name to `Match`) and forwards each request to the right match.
- **Match:** One object per match. It owns the board, the two players, their listeners and the current turn, and it is the only place where the game rules are applied.
- **GameClientListener (Interface):** Remote callback interface residing on the client to receive asynchronous updates (`notifyMoveMade`, `notifyGameOver`, `notifyGameStarted`).
- **GameClientListenerImpl:** Client-side implementation of the callback interface; it forwards every event to the JavaFX thread.
- **GameController:** JavaFX frontend controller handling user input, the lobby (with separate *Create Match* and *Join Match* actions) and board updates.

2.2 Concurrency Design

One object per match. All the state of a match lives in a single `Match` object, and every method that reads or changes it (`join`, `makeMove`) is `synchronized`. Each match is therefore a *monitor* with its own lock: operations on the same match are executed one at a time, so checks such as "is the game full?" or "is it this player's turn?" and the updates that follow them are atomic, while different matches never block each other.

Atomic creation. The matches are stored by `GameEngineImpl` in a `ConcurrentHashMap` and created with `putIfAbsent`. If two clients create a match with the same name at the same time, exactly one of them succeeds and the other receives "Game already exists!".

Callbacks outside the lock. Calling a client from inside a critical section is dangerous: a slow or unreachable client would keep the match locked for every other request, and an exception thrown halfway through would leave the match half-updated. For this reason `Match` first completes the state change (board, winner check, turn switch) and only then *queues* the notifications on a single thread owned by the match (`notifier`). The remote calls run on that thread, without holding the lock. Because there is exactly

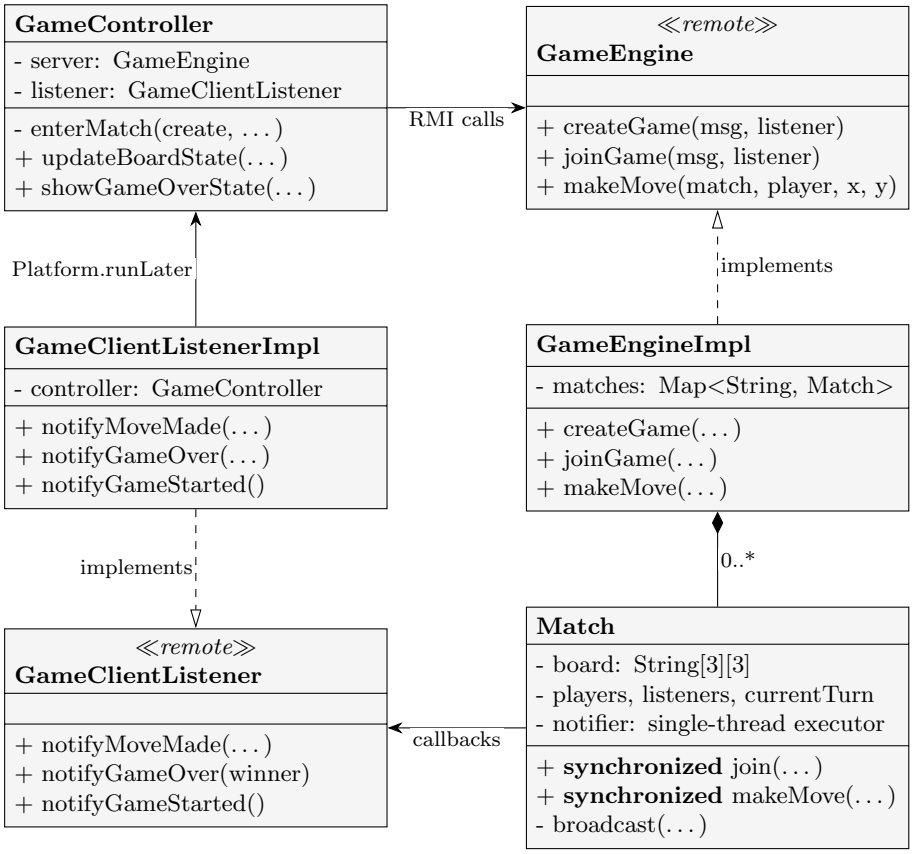


Figure 1: Class diagram. Left: client side; right: server side.

one notifier thread per match, events reach each client in the same order in which they happened.

Unreachable clients. If a callback fails with a `RemoteException`, the notifier marks the match as over, tells the remaining player that they won, and removes the match from the server. The game state is never left inconsistent, and no other match is affected.

3 Interaction Flow

The lifecycle of a game session follows these distributed steps (Figure 2):

1. **Connection & Lobby:** When the client starts, it looks up the `"GameEngine"` in the RMI registry (`localhost:1099`) and exports its own `GameClientListener`. In the lobby the user types a username and a match name and chooses *Create Match* or *Join Match*.

2. **Match Creation:** `createGame(Message, listener)` creates a new `Match` with an empty 3×3 board and the creator as first player, and registers it under its name. The creator then waits for an opponent.

3. **Match Joining:** `joinGame(Message, listener)` looks up the match by name and adds the second player. It fails if the match does not exist, is full, or the username is already used in that match. When the second player joins, the match triggers `notifyGameStarted()` on both clients.

4. **Move Execution Loop:**

   - A player clicks a cell; the client invokes `server.makeMove(...)`.
   - The match validates turn, coordinates and cell inside its synchronized method, updates the board, checks for a win or a draw and switches the turn.
   - The match queues `notifyMoveMade(...)` for both players.

5. **Termination:** When the game is won or drawn, the match queues `notifyGameOver(...)`, the clients lock their boards, and the match is removed from the server, so its name can be reused. If one of the clients cannot be reached, the other player is declared the winner.

4 Implementation Highlights

- **Thread Safety on Server:** The directory of matches is a `ConcurrentHashMap`, and new matches are added with `putIfAbsent`. All game state is encapsulated in `Match`, whose `synchronized` methods make every check-and-update atomic.

- **Ordered, Non-blocking Callbacks:** Each `Match` owns a single-thread `ExecutorService` that delivers callbacks in order and outside the lock. Every remote call is wrapped in its own `try/catch`, so the failure of one client cannot prevent the other client from being notified.

- **UI Thread Concurrency:** On the client, every callback is handed to the JavaFX application thread with `Platform.runLater(...)`, to avoid multi-threading UI exceptions.

- **Error Reporting:** Game errors ("Game is full!", "It is not your turn!", ...) are thrown by the server as `RemoteException`s and reach the client wrapped in an RMI `ServerException`; the client unwraps them and shows only the server's message.

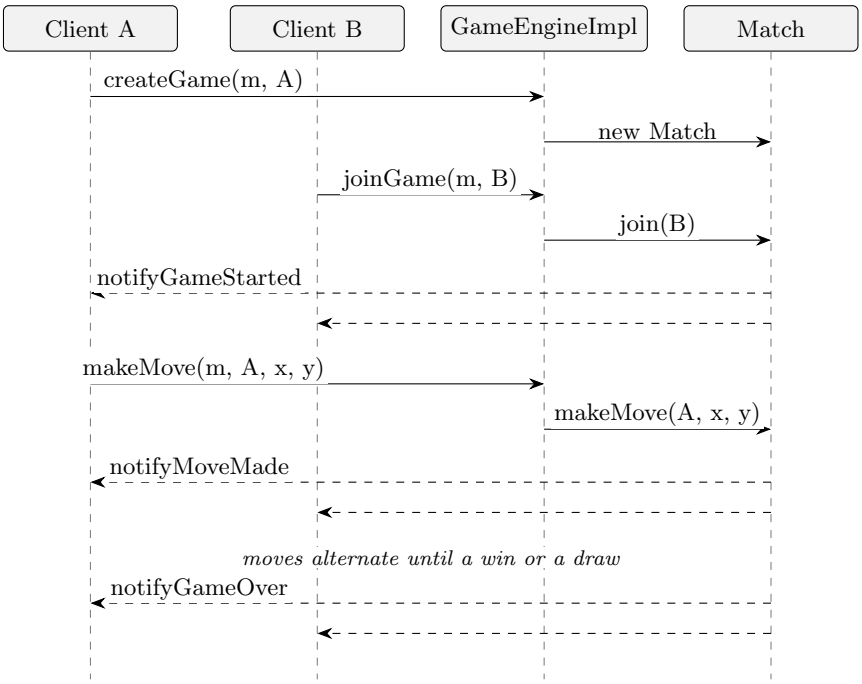


Figure 2: Interaction of two clients. Solid arrows are remote calls from the clients; dashed arrows are callbacks, sent by the match's notifier thread after the lock has been released.